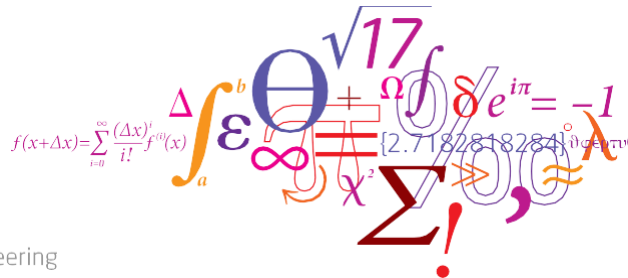


Lecture 8 - Clustering

with contributions from: Fang Zhao, Haizheng Zhang, Moshe Ben-Akiva



Outline

- Introduction
- Similarity measures
- Clustering algorithms
 - K-means
 - Density-Based clustering
- Evaluation
- Appendix
 - Gaussian Mixture
 - Hierarchical

Learning Objectives

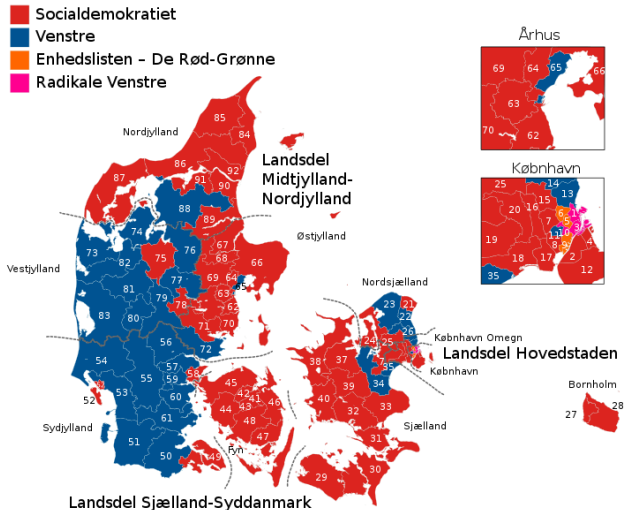
- Define Similarity measures and their impact clustering
- Explain the K-means algorithm and its basic principles
- Describe the DBSCAN algorithm and its approach to clustering
- Discuss the strengths and limitations of DBSCAN and K-means
- Implement K-means/DBSCAN clustering using Python
- Evaluate clustering results

What is Clustering?

- Clustering means grouping n observations into clusters (partitions).
- There is no dependent variable, n
 - We group observations, z_j , by their similarity
 - *unsupervised learning*
- **Exploratory** technique to discover interesting relationships in data.

Clustering Applications: Voting

- You can use clustering to find people with similar interests of needs;
- For example, people in the larger cities may have different interests from differently than those living in smaller urban areas or outside of it
- If we would know how individuals vote in advance, we may use classification. But (fortunately) we cannot.



- Comparing patterns (bike sharing data)
- Challenge: group stations according to similar demand patterns
- Example: *Hubway rentals dataset*

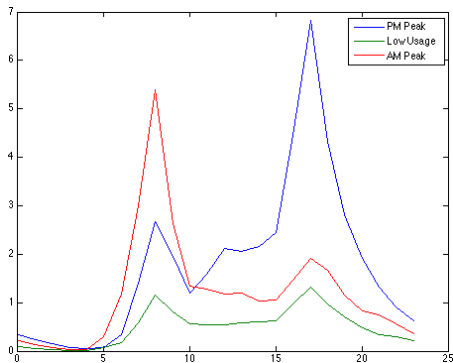


Figure: Example 2

Clustering Applications: Marketing

- **Customer segmentation** based on brand loyalty and price sensitivity scores.

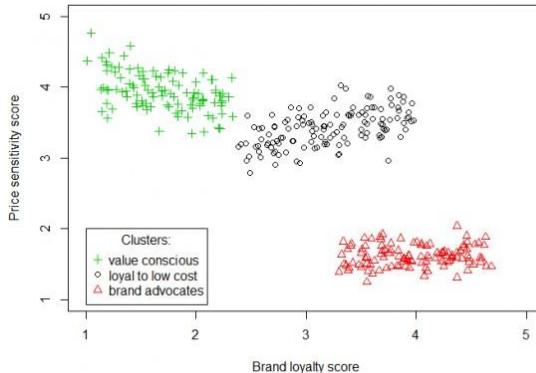


Figure: Example 3

- How similar are two observations?
 - Geographical distance
 - Vehicle color
 - Vehicle type
 - Vehicle brand
 - Engine type
 - Engine power
 - ...



Figure: Vehicles as items for cluster analysis

Similarity Measures



Similarity Measures

Comparing two vectors, $\mathbf{z}_k$ and $\mathbf{z}_j$, with r variables

- *Numerical data:*

- Euclidean distance

(default metric in Sklearn)

$$d(\mathbf{z}_k, \mathbf{z}_j) = \sqrt{\sum_{i=1}^r (z_{ki} - z_{ji})^2}$$

- Manhattan distance

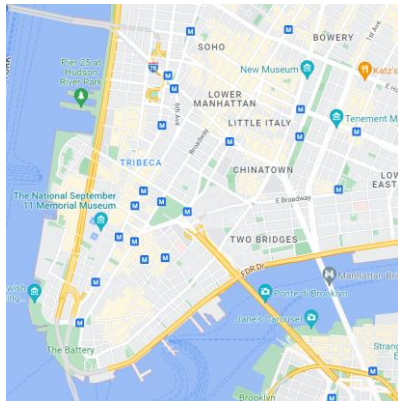
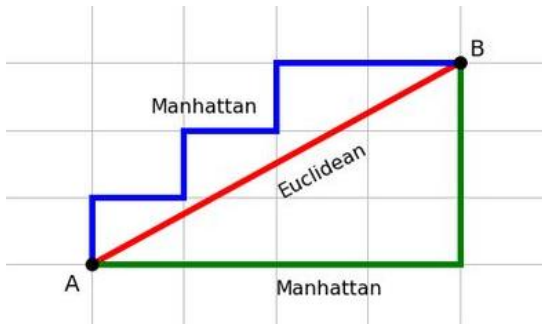
$$d(\mathbf{z}_k, \mathbf{z}_j) = \sum_{i=1}^r |z_{ki} - z_{ji}|$$

- Minkowski distance

$$d(\mathbf{z}_k, \mathbf{z}_j) = \left[\sum_{i=1}^r |z_{ki} - z_{ji}|^m \right]^{1/m}$$

Similarity Measures

Difference between Manhattan distance and Euclidean distance



Similarity Measures

- *Categorical data*:
 - Based on **presence or absence** of certain characteristics (*binary variables*).

	Variables				
	1	2	3	4	5
Item i	1	0	0	1	1
Item k	1	1	0	1	0

- **Contingency table**: variable matches and mismatches between observations (items) i and k .

	Item k		
	1	0	Totals
Item i 1	a	b	$a + b$
Item i 0	c	d	$c + d$
Totals	$a + c$	$b + d$	$p = a + b + c + d$

(12-7)

Similarity Measures

- **Contingency table:** variable matches and mismatches between observations (items) i and k .

		Item k		
		1	0	Totals
Item i	1	a	b	$a + b$
	0	c	d	$c + d$
Totals		$a + c$	$b + d$	$p = a + b + c + d$

(12-7)

- Distance measure for **symmetric** binary variables, $ds(i, k)$

$$ds(i, k) = \frac{b + c}{a + b + c + d}$$

- Distance measure for **asymmetric** binary variables, $da(i, k)$

$$da(i, k) = \frac{b + c}{a + b + c}$$

- Jaccard **similarity** coefficient, $J(i, k)$

$$J(i, k) = \frac{a}{a + b + c}$$

Similarity Measures

- **Contingency table:** variable matches and mismatches between observations (items) i and k .

		Item k		
		1	0	Totals
Item i	1	a	b	$a + b$
	0	c	d	$c + d$
Totals		$a + c$	$b + d$	$p = a + b + c + d$

(12-7)

- Distance measure for **symmetric** binary variables, $ds(i, k)$

$$ds(i, k) = \frac{b + c}{a + b + c + d}$$

- Distance measure for **asymmetric** binary variables, $da(i, k)$

$$da(i, k) = \frac{b + c}{a + b + c}$$

- Jaccard **similarity** coefficient, $J(i, k)$ (only cares of the 1s, not the zeros)

$$J(i, k) = \frac{a}{a + b + c}$$

Similarity Measures:

- Careful with multiple dimensions with different scales

- Normalization:

$$z_{new} = \frac{z - z_{min}}{z_{max} - z_{min}}, z_{new} \in [0,1]$$

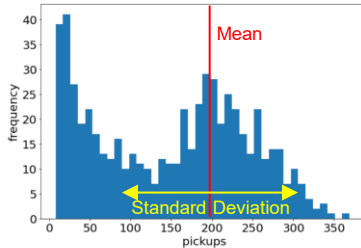
- Standardization:

$$z_{new} = \frac{z - \bar{z}}{s_z}, z_{new} \sim D(0,1)$$

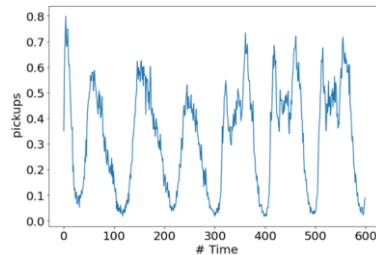
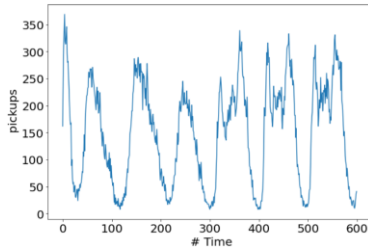
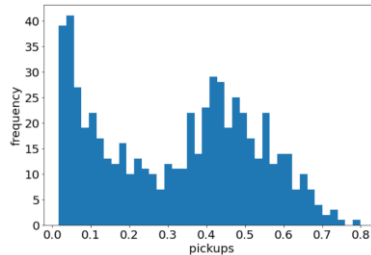
Similarity Measures:

- An Example. NY Taxi Pickups

Raw data



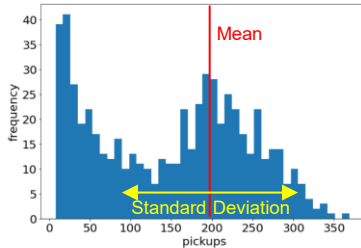
Normalization



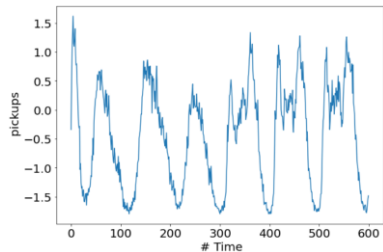
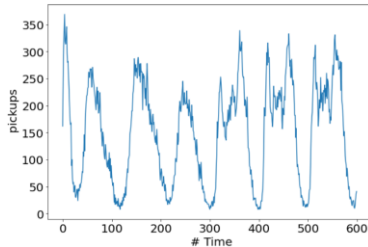
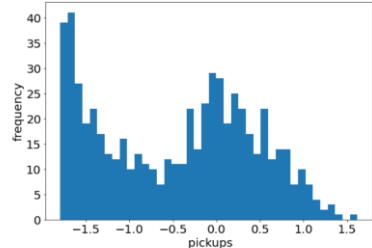
Similarity Measures:

- An Example. NY Taxi Pickups

Raw data



Standardization



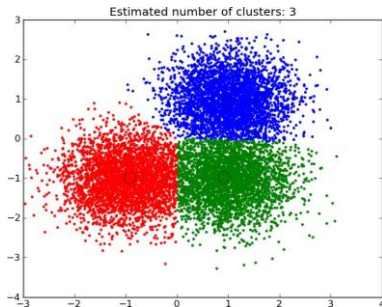
Some notes about distance metrics

- Choice of similarity measure:
 - May lead to **different groupings**.
 - **Subjective** and domain-dependent
 - Dependent on the **variable type** (discrete, continuous, binary).
 - Dependent on the **scale of measurement**
 - Sometimes we need a combination of different types of metrics

Clustering



- *K*-means clustering
- Density-based clustering
- Distribution-based clustering (Appendix)
- Hierarchical clustering (Appendix)

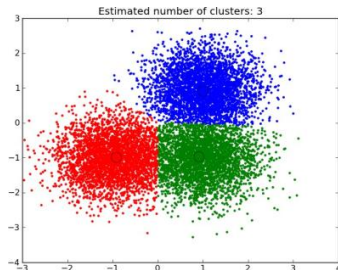


Source: www.scikit-learn.org

Figure: Clustering result ($K = 3$)

K-means Clustering: Definitions

- Group observations into K clusters (K is an input)
- A cluster centroid is the mean of observations assigned to that cluster
- An observation belongs to the cluster with the closest centroid

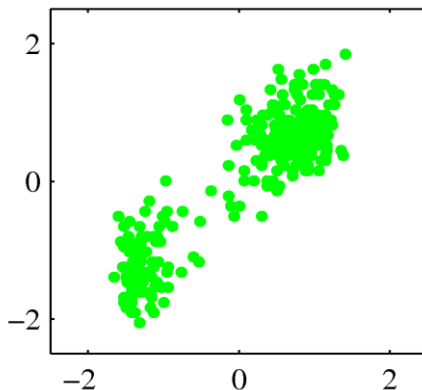
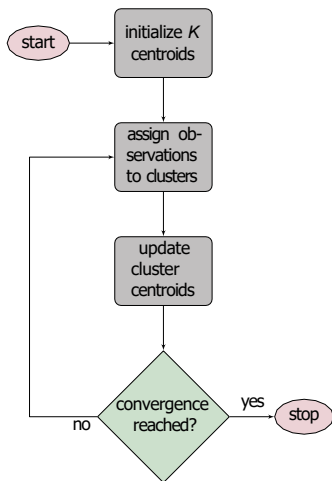


Source: www.scikit-learn.org

K-means Clustering: Algorithm

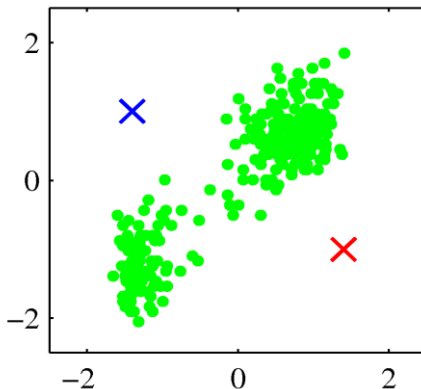
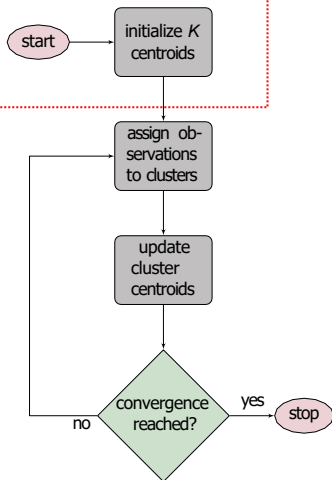
- K -means starts by **initialization** of K centroids, e.g., random pick of K observations, user determined
- **Iterative procedure** keeps reassigning observations to their closest clusters and recomputing the cluster centroids (for changed clusters).
- Clustering ends when **convergence** is reached (e.g. no new reassignments observed).
- Clustering result can be **sensitive** to the initial partitioning (so, it is necessary to rerun the algorithm with different initial partitions).

K-means Clustering: Algorithm

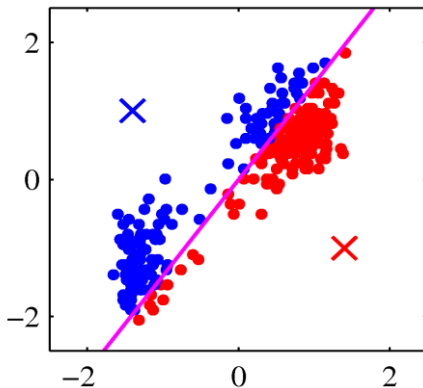
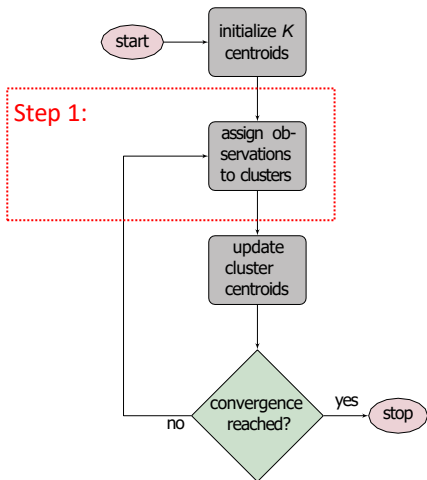


K-means Clustering: Algorithm

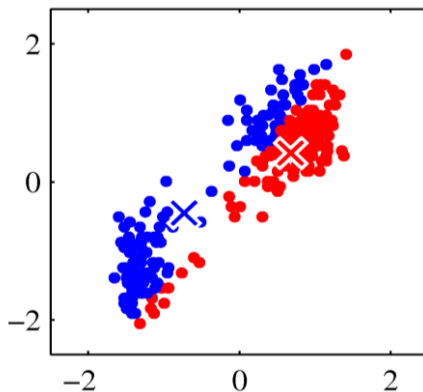
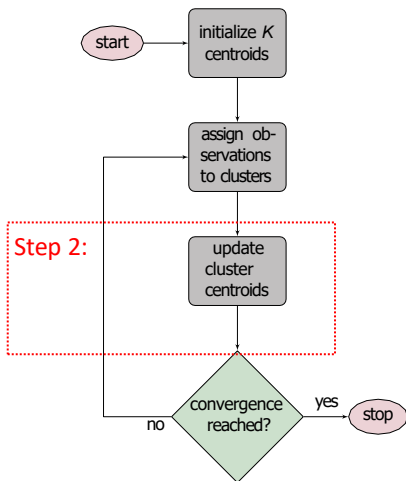
Step 0



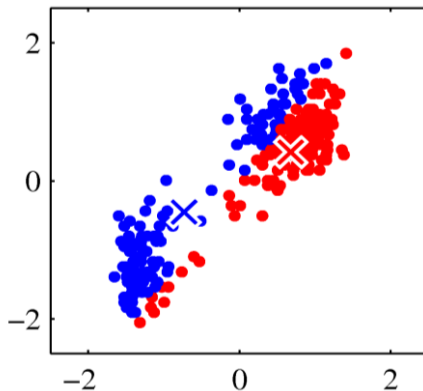
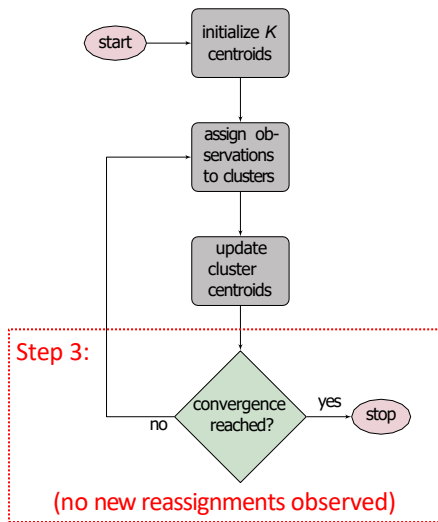
K-means Clustering: Algorithm



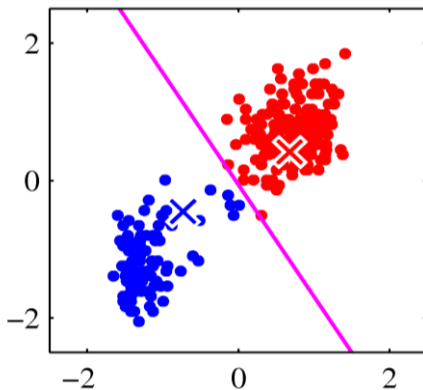
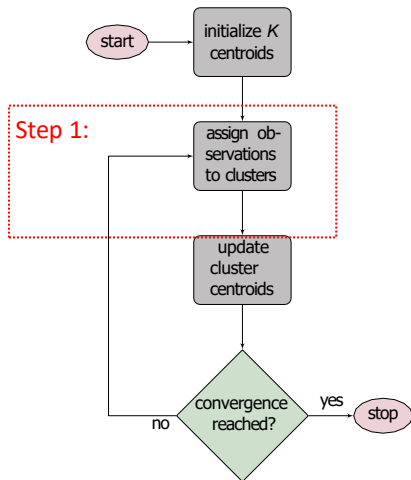
K-means Clustering: Algorithm



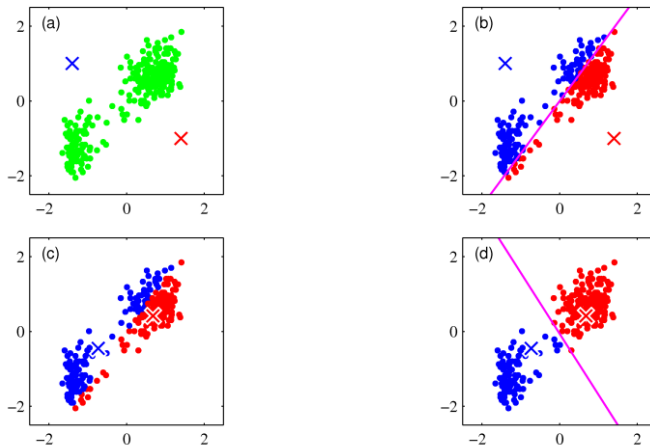
K-means Clustering: Algorithm



K-means Clustering: Algorithm

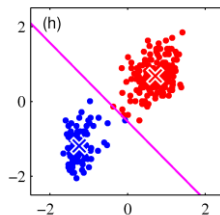
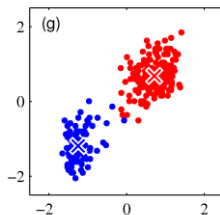
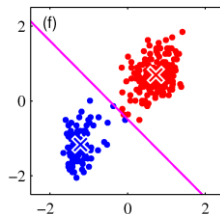
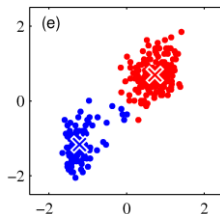


K-means Clustering: Illustration

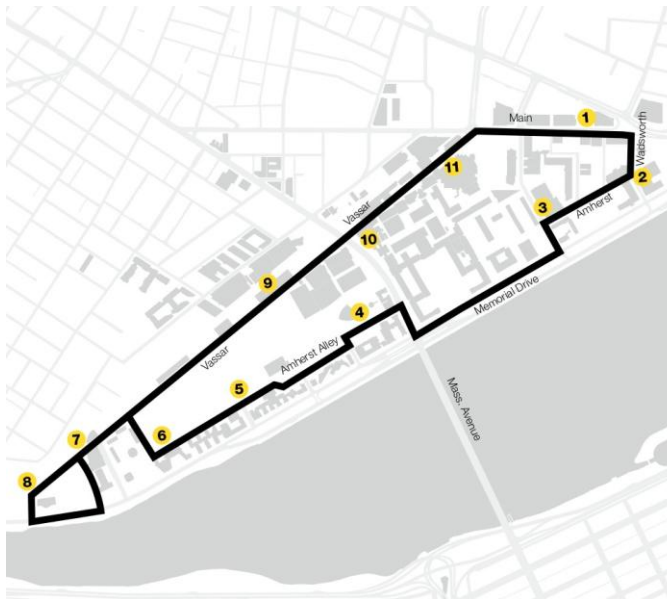


Source: Cristopher M. Bishop, *Pattern Recognition and Machine Learning*

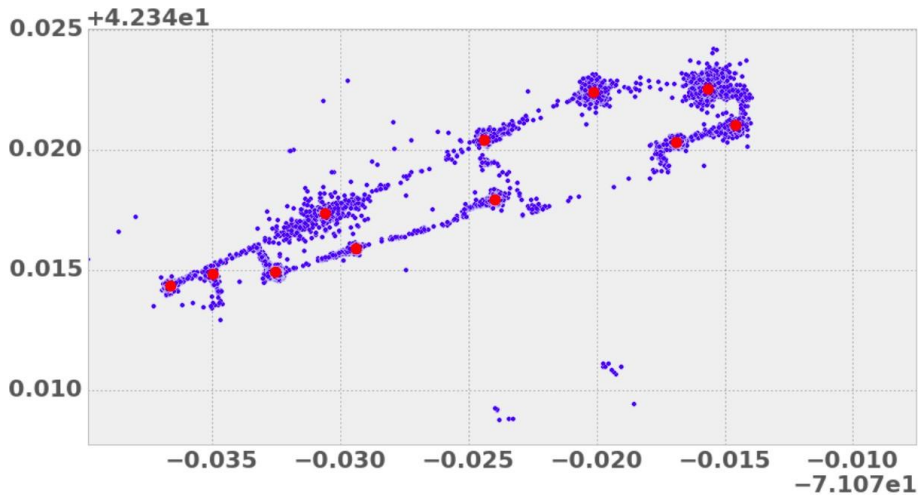
K-means Clustering: Illustration



Source: Cristopher M. Bishop, *Pattern Recognition and Machine Learning*

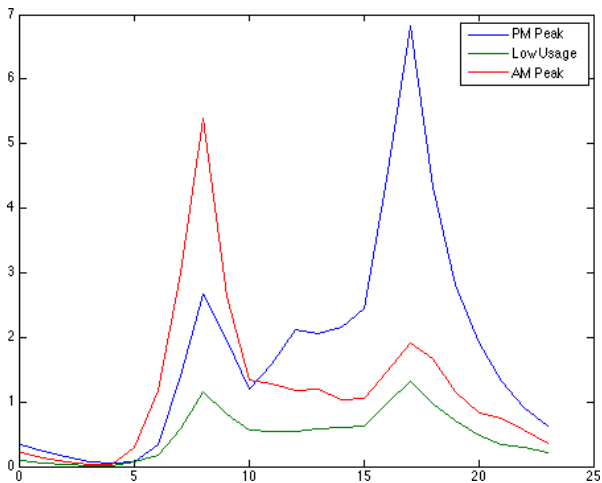


- 1 Kendall Square T
- 2 Amherst / Wadsworth
- 3 Media Lab
- 4 Amherst @ Kresge
- 5 Burton House
- 6 Tang / Westgate
- 7 W92 @ Amesbury St
- 8 W98 @ Vassar St
- 9 Simmons Hall
- 10 Vassar / Mass Ave
- 11 Stata Center



Purple dots: GPS points from buses; Red dots: centroids

- Comparing patterns
- Challenge: group stations according to similar demand patterns



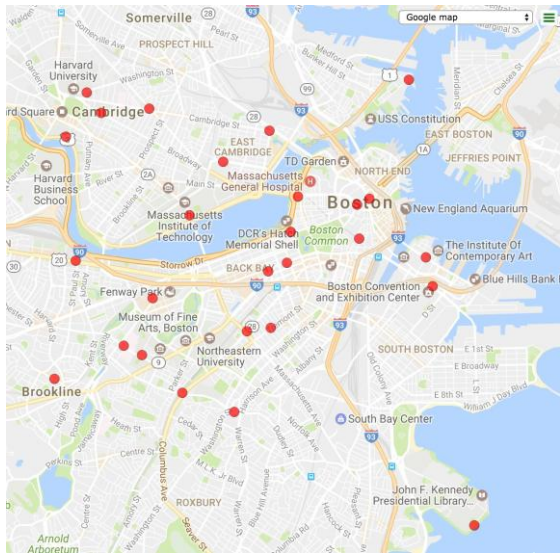


Figure: AM Peak

Clustering Hubway Rentals

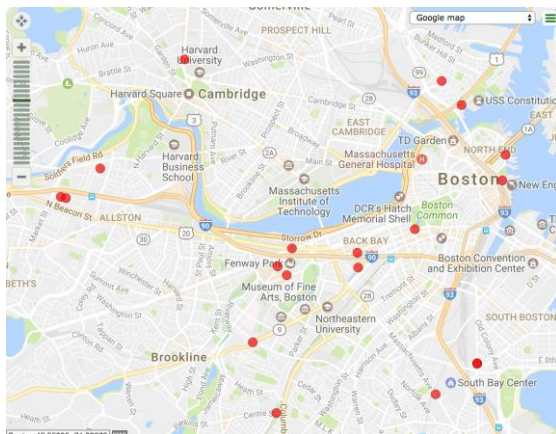


Figure: PM Peak

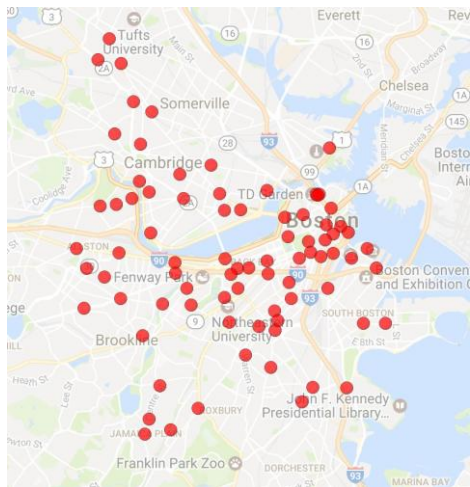


Figure: Low Usage

Practical considerations

Advantages

- Intuitive
- Computationally very fast
- Very easy to implement/tweak (e.g. for different similarity metrics)

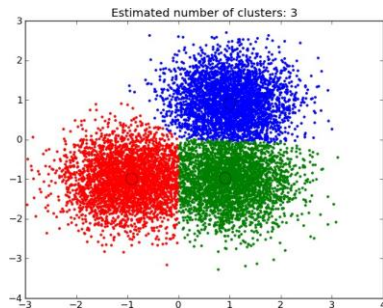
Disadvantages

- Need to choose a fixed K (arbitrarily)
- Sensitive to outlayers
- Initialization affects results
- Cluster tends to be spherical
- Exclusivity: one observation $\rightarrow$ one cluster

Playtime!

- Open the "6. Clustering.ipynb" notebook
- Do Part 1
- Estimated duration: 45 min

- K -means demands choice of K , and enforce symmetry
- Clusters are symmetric
- Classify all data points, including outliers.
- Density-based clustering:
 - Number of clusters need not be specified
 - Based on inclusion rule (ϵ -neighbourhood)
 - Penalises sparsity (robust to noise/outliers)
 - Allows for arbitrary cluster shapes

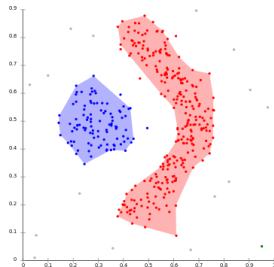


Source: www.scikit-learn.org

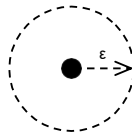
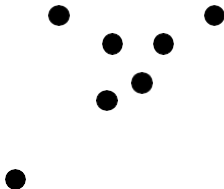
Figure: Clustering result ($K = 3$)

DBSCAN Algorithm: Definitions

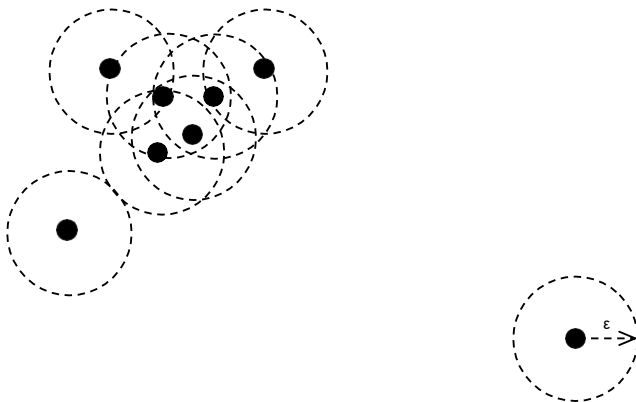
- Parameters:
 - $MinPts$: minimum number of points to form a **core**
 - ε : radius of reachability of a point
- Definitions:
 - A **core point** has at least $MinPts$ points in its ε -neighborhood (N_ε).
 - A cluster is a chain of core points and **border points** (on the edge of the cluster).



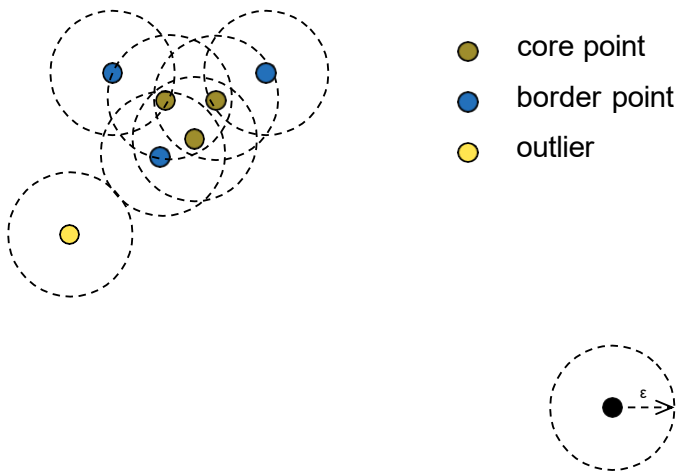
DBSCAN Algorithm: Illustration (MinPts=3)



DBSCAN Algorithm: Illustration (MinPts=3)



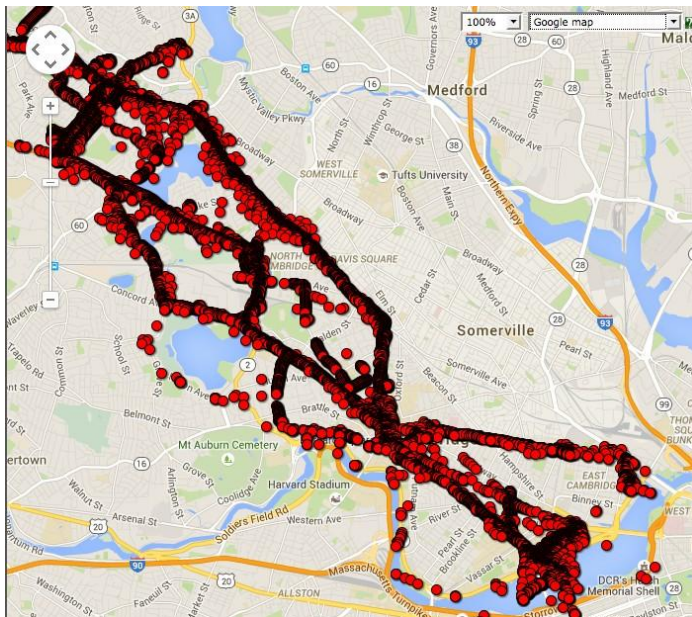
DBSCAN Algorithm: Illustration (MinPts=3)



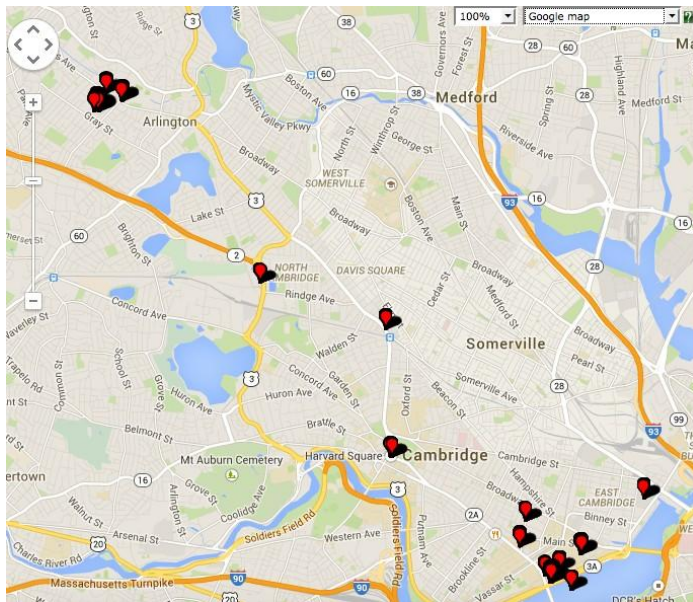
DBSCAN Algorithm: Illustration (MinPts=3)



DBSCAN Algorithm: Example

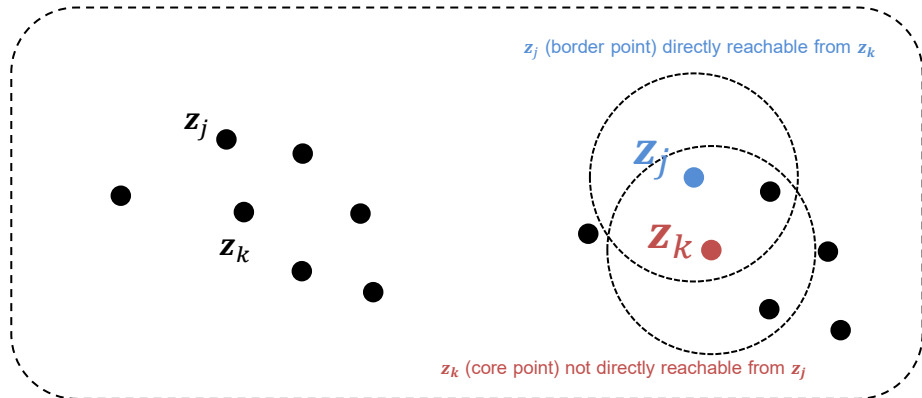


DBSCAN Algorithm: Example



DBSCAN Algorithm: Formal definitions

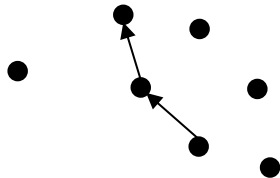
- A point z_j is *directly density-reachable* from a point z_k if:
 - 1 $z_j \in N_\epsilon(z_k)$ and
 - 2 $|N_\epsilon(z_k)| \geq \text{MinPts}$ (core point condition)
- Directly density-reachable is symmetric for pairs of core points. In general, however, it is not symmetric if one core point and one border point are involved



DBSCAN Algorithm: Formal definitions

- A point z_j is *density-reachable* from a point z_k if:
 - There is a chain of points $z_1, z_2, \dots, z_q$, $z_1 = z_j, z_q = z_k$ such that z_{l+1} is directly density-reachable from z_l .
 - Density-reachability is an extension of direct density-reachability. This relation is transitive, but it is not symmetric. The figure below depicts the relations of some sample points and, in particular, the asymmetric case.

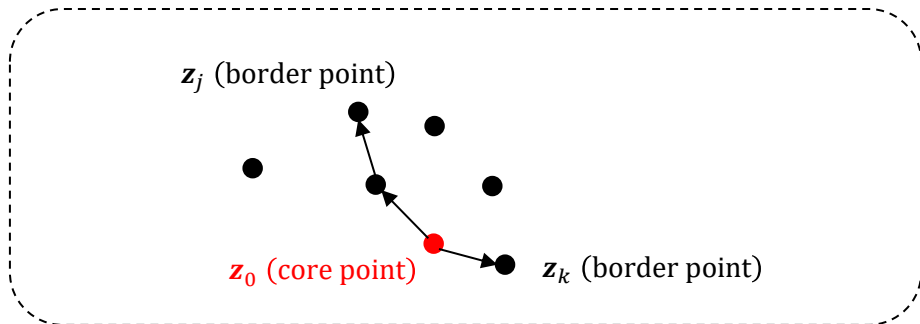
z_j (border point) density reachable from z_k



z_k (core point) not density reachable from z_k

DBSCAN Algorithm: Formal definitions

- A point z_j is *density-connected* to a point z_k if:
 - There is a point z_0 such that both z_j and z_k are density-reachable from z_0 .



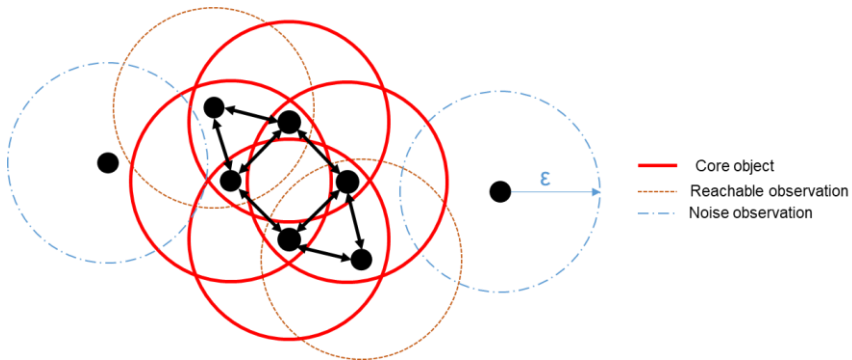
DBSCAN Algorithm: Formal definitions

- A point $\mathbf{z}_j$ is *directly density-reachable* from a point $\mathbf{z}_k$ if:
 - ① $\mathbf{z}_j \in N_\varepsilon(\mathbf{z}_k)$ and
 - ② $|N_\varepsilon(\mathbf{z}_k)| \geq MinPts$ (core point condition)
- A point $\mathbf{z}_j$ is *density-reachable* from a point $\mathbf{z}_k$ if:
 - There is a chain of points $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_q$, $\mathbf{z}_1 = \mathbf{z}_j$, $\mathbf{z}_q = \mathbf{z}_k$ such that $\mathbf{z}_{l+1}$ is directly density-reachable from $\mathbf{z}_l$.
- A point $\mathbf{z}_j$ is *density-connected* to a point $\mathbf{z}_k$ if:
 - There is a point $\mathbf{z}_0$ such that both $\mathbf{z}_j$ and $\mathbf{z}_k$ are density-reachable from $\mathbf{z}_0$.

DBSCAN Algorithm: Formal definitions

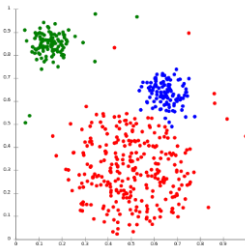
- A **cluster** C is a non-empty subset of points satisfying conditions:
 - 1 $\forall \mathbf{z}_j, \mathbf{z}_k$: if $\mathbf{z}_j \in C$ and $\mathbf{z}_k$ is density-reachable from $\mathbf{z}_j$, then $\mathbf{z}_k \in C$.
(Maximality)
 - 2 $\forall \mathbf{z}_j, \mathbf{z}_k \in C$: $\mathbf{z}_j$ is density-connected to $\mathbf{z}_k$. (Connectivity)
- Short: A cluster is a maximal set of density-connected points.
- **Noise** is the set of points not belonging to any cluster.

DBSCAN Algorithm: Formal definitions



Practical Considerations

- Varying cluster densities
 - We may need different cluster densities



Source: Wikipedia

- DBSCAN may be demanding in terms of memory
- Trivial to understand in spatial domain, not obvious in others (particularly $\geq 3D$)

Playtime!

- Open the "6. Clustering.ipynb" notebook
- Do Part 2
- Estimated duration: 30 min

Validation



Clustering Evaluation

- Good clustering should:
 - Minimize **within-cluster** (inter-cluster) variability (**W**).
 - Maximize **between-cluster** (intra-cluster) variability (**B**).
- Measures: **distortion**, **silhouette**

- Distortion, *dist*:

$$dist = \sum_{\substack{k \text{ centroids,} \\ j \text{ vectros}}} d\left(\mathbf{z}_j^{(k)}, \bar{\mathbf{z}}^{(k)}\right)^2$$

- Where $\mathbf{z}_j^{(k)}$ are vectors that belong to cluster k ,
- The centroid of cluster k is $\bar{\mathbf{z}}^{(k)}$
- A *good* clustering will have low *dist*
- d is a distance function
- Distortion measures within cluster variability **W**

- Silhouette of observation $\mathbf{z}_j$, $s(\mathbf{z}_j)$:

$$s(\mathbf{z}_j) = \frac{b(\mathbf{z}_j) - a(\mathbf{z}_j)}{\max(b(\mathbf{z}_j), a(\mathbf{z}_j))}$$

- $a(\mathbf{z}_j)$ = average distance between $\mathbf{z}_j$ and *all* other elements of its cluster (intra-cluster distance)
- $b(\mathbf{z}_j)$ = average distance between $\mathbf{z}_j$ and *all* elements of the second nearest cluster.
- Silhouette measures both within cluster and between cluster variability

Silhouette

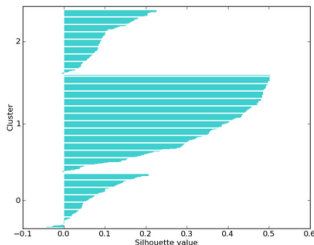
- Measures how well an observation fits a cluster

$$-1 < s(\mathbf{z}_j) < 1$$

- We want $a(\mathbf{z}_j)$ to be small and $b(\mathbf{z}_j)$ to be large:

$$a(\mathbf{z}_j) \ll b(\mathbf{z}_j) \Rightarrow s(\mathbf{z}_j) \rightarrow 1$$

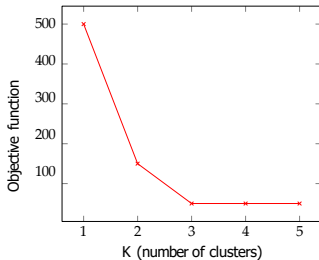
- A common visualization:



- The silhouette for all data is the average of all $s(\mathbf{z}_j)$

The Elbow curve

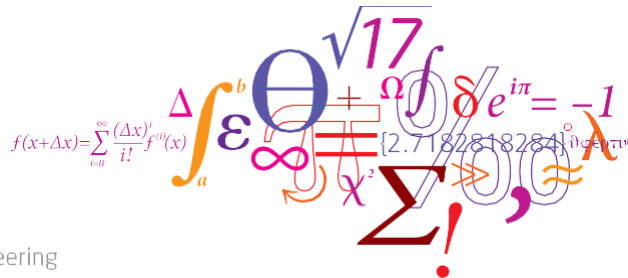
- Algorithms may demand parameters (e.g. K in K -means and mixture models, $MinPts$ in DBSCAN)
- Plot objective function values against different parameter choices (e.g. K)
- Choose parameter at the elbow-point (e.g. $K = 3$)



Reading list (in red, same examples/material as in class)

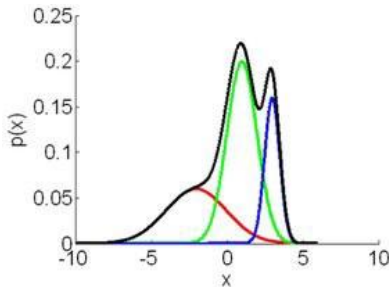
- *Data-Science from Scratch*. Chapter 20 (K-means, elbow curve, Hierarchical clustering)
- *Pattern Recognition and Machine Learning*. Chapter 10 (K-means)
- Johnson, R.A. and Wichern, D.W. (2002). *Applied Multivariate Statistical Analysis*, 5th ed. Prentice Hall Sections 12.1, 12.2, 12.3 and 12.4
https://archive.org/details/epdf.pub_applied-multivariate-statistical-analysis-6th-edit/page/n95/mode/2up (distance and similarity coefficients, hierarchical clustering)
- *Gaussian Mixture Model clearly explained: [Gaussian Mixture Model: A Comprehensive Guide to Understanding and Implementing GMM from Scratch | Towards Data Science](#)*
- Ester, M., Kriegel, H.-P., Sander, J. and Xu, Xiaowei. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.
<https://aaai.org/papers/037-a-density-based-algorithm-for-discovering-clusters-in-large-spatial-databases-with-noise/>

Appendix



Distribution based clustering

- Assume data is generated by an underlying process
 - There are K distributions, $f_k(\mathbf{Z}; \boldsymbol{\alpha}_K)$
 - Each observation $\mathbf{z}_j$ generated by **one** f_k , according to probability $p(k) = \theta_k$
 - Example with univariate Gaussians ($K = 3$)



From <http://hdl.handle.net/10380/3218>

Mixture models

- We want to:
 - Estimate the underlying distributions

$$f_k(\mathbf{Z}; \boldsymbol{\alpha}_k)$$

- Estimate the probabilities $p(k) = \theta_k$
- For each observation, $\mathbf{z}_j$, determine its generating distribution (i.e. its cluster)

$$\begin{aligned} p(k | \mathbf{z}_j; \boldsymbol{\alpha}_k) &= \frac{p(k)f_k(\mathbf{z}_j|k)}{f(\mathbf{z}_j)} = \frac{\theta_k f_k(\mathbf{z}_j; \boldsymbol{\alpha}_k)}{f(\mathbf{z}_j)} = \\ &= \frac{\theta_k f_k(\mathbf{z}_j; \boldsymbol{\alpha}_k)}{\sum_{k=1}^K \theta_k f_k(\mathbf{z}_j; \boldsymbol{\alpha}_k)} \end{aligned}$$

Mixture models

- We do not know the parameters, α_k or θ_k
- We assume the form of all f_k (e.g. each f_k is a Gaussian distribution)
- We know that:

$$f(\mathbf{Z}) = \sum_{k=1}^K \theta_k f_k(\mathbf{Z}; \alpha_k)$$

with $\sum_{k=1}^K \theta_k = 1$

Expectation-Maximization algorithm (EM)

1 Initialize $\alpha_k^{(0)}$ and $\theta_k^{(0)}$

2 **E-step:** Calculate $\gamma_{jk}^{(0)} = p(k | \mathbf{z}_j; \alpha_k^{(0)})$ through Bayes theorem:

$$\gamma_{jk}^{(0)} = p(k | \mathbf{z}_j; \alpha_k^{(0)}) = \frac{\theta_k^{(0)} f_k(\mathbf{z}_j; \alpha_k^{(0)})}{\sum_{k=1}^K \theta_k^{(0)} f_k(\mathbf{z}_j; \alpha_k^{(0)})}$$

Update $\theta_k^{(t+1)} = \frac{n_k^{(t)}}{n}$ with $n_k^{(t)} = \sum_{j=1}^n \gamma_{jk}^{(t)}$

3 **M-step:** Re-estimate the parameters $\alpha_k^{(t+1)}$ by maximizing the log likelihood function:

$$LL(t+1) = \sum_{j=1}^n \log \left\{ \sum_{k=1}^K \theta_k^{(t+1)} f_k(\mathbf{z}_j; \alpha_k^{(t+1)}) \right\}$$

4 Loop to step 2 until $LL(t+1) - LL(t) < \varepsilon$

EM with Gaussian mixture

1 Initialize mean, μ_k and covariance, Σ_k , of K Gaussians; initialize θ_k

2 E-step: $\gamma_{jk} = p(k | \mathbf{z}_j, \mu_k, \Sigma_k) = \frac{\theta_k N(\mathbf{z}_j; \mu_k, \Sigma_k)}{\sum_{k=1}^K \theta_k N(\mathbf{z}_j; \mu_k, \Sigma_k)}$

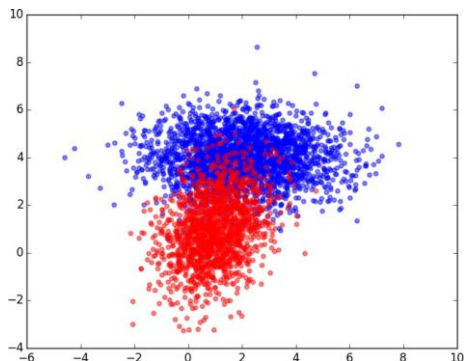
4 M-step: update parameters

$$\mu_k^{new} = \frac{\sum_{j=1}^n \gamma_{jk} \mathbf{z}_j}{\sum_{j=1}^n \gamma_{jk}} \quad \Sigma_k^{new} = \frac{1}{n_k} \sum_{j=1}^n \gamma_{jk} (\mathbf{z}_j - \mu_k^{new})(\mathbf{z}_j - \mu_k^{new})'$$

6 Loop until μ and Σ converge

Example: a bivariate mixture

$$f(\mathbf{Z}) = \theta_1 N(\mathbf{Z}; \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + (1 - \theta_1) N(\mathbf{Z}; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$$



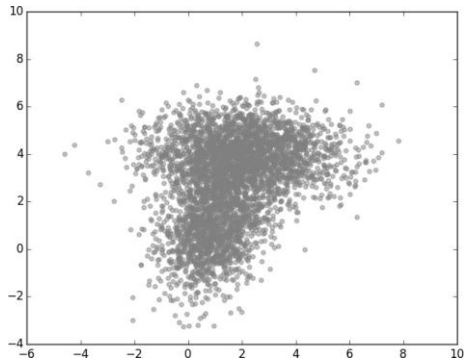
$$\theta_1 = 0.6$$

$$\boldsymbol{\mu}_1 = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \quad \boldsymbol{\Sigma}_1 = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\boldsymbol{\mu}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \boldsymbol{\Sigma}_2 = \begin{bmatrix} 1 & 0.4 \\ 0.4 & 2 \end{bmatrix}$$

Example: a bivariate mixture

What the algorithm “sees”



- Results with Expectation-Maximization (Python sklearn):

$$\theta_1 = 0.6$$

$$\mu_1 = \begin{bmatrix} 1.972 \\ 3.995 \end{bmatrix} \quad \Sigma_1 = \begin{bmatrix} 2.943 & 0.034 \\ 0.034 & 1.039 \end{bmatrix}$$

$$\mu_2 = \begin{bmatrix} 1 \\ 0.933 \end{bmatrix} \quad \Sigma_2 = \begin{bmatrix} 1.070 & 0.46 \\ 0.46 & 2.052 \end{bmatrix}$$

Practical considerations

Advantages

- Probabilistic cluster membership
- Very easy to tweak (e.g. for different distributions)
- Computationally fast

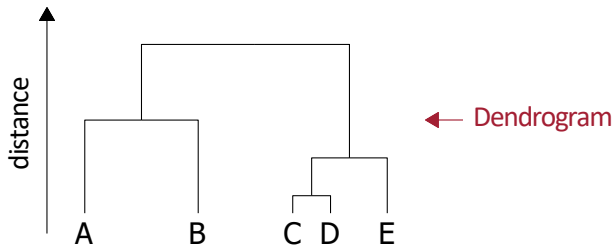
Disadvantages

- Need to choose a fixed K (arbitrarily)
- Initialization affects results

- In previous algorithms, choice of K is very restrictive
- Data can often be grouped at different levels (multiple values for K ?)
- The idea is to build a binary tree of the data that successively merges similar groups
- Visualizing this tree can provide a useful summary of the data

Hierarchical Clustering

- Hierarchical clustering techniques can be:
 - **Agglomerative** (*bottom-up*) - from individual observations cluster to larger clusters
 - **Divisive** (*top-down*) - from one large cluster into smaller clusters



Hierarchical Clustering: Agglomerative

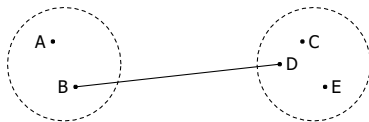
- Agglomerative methods in each step merge **two most similar** (closest) clusters.
- Distance between clusters (**linkage**)?
 - *Single linkage* - minimum distance or nearest neighbor (2 closest border points)
 - *Complete linkage* - maximum distance or farthest neighbor (2 farthest border points)
 - *Average linkage* - average distance (all to all)

Hierarchical Clustering: Linkage

- Single linkage

$$d_{(AB)(CDE)} = \min(d_{AC}, d_{AD}, d_{AE}, d_{BC}, d_{BD}, d_{BE})$$

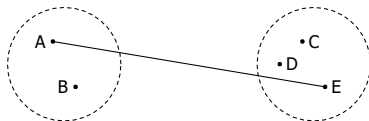
$$= d_{BD}$$



- Complete linkage

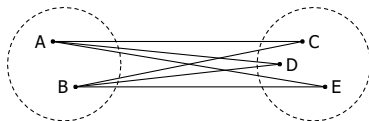
$$d_{(AB)(CDE)} = \max(d_{AC}, d_{AD}, d_{AE}, d_{BC}, d_{BD}, d_{BE})$$

$$= d_{AE}$$

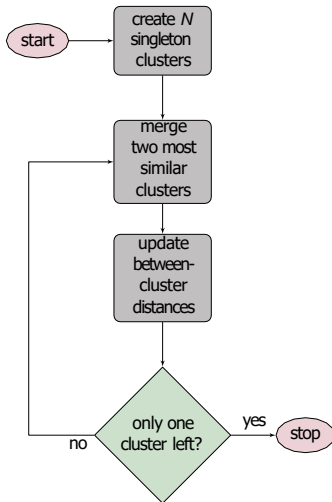


- Average linkage

$$d_{(AB)(CDE)} = \frac{d_{AC} + d_{AD} + d_{AE} + d_{BC} + d_{BD} + d_{BE}}{6}$$



Hierarchical Clustering: Flowchart



Single linkage: Illustration

$$\{d_{ik}\} = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & & & & \\ 9 & 0 & & & \\ 3 & 7 & 0 & & \\ 6 & 5 & 9 & 0 & \\ 11 & 10 & \textcircled{2} & 8 & 0 \end{bmatrix} \end{matrix}$$

$$\begin{matrix} & \begin{matrix} (CE) & A & B & D \end{matrix} \\ \begin{matrix} (CE) \\ A \\ B \\ D \end{matrix} & \begin{bmatrix} 0 & & & \\ \textcircled{3} & 0 & & \\ 7 & 9 & 0 & \\ 8 & 6 & 5 & 0 \end{bmatrix} \end{matrix}$$

$$\min(d_{ik}) = d_{EC} = 2$$

- Merge the closest clusters (C and E).
- Recompute cluster distances (single linkage):
 $d_{(CE)A} = \min\{d_{CA}, d_{EA}\} = 3$
 $d_{(CE)B} = \min\{d_{CB}, d_{EB}\} = 7$
 $d_{(CE)D} = \min\{d_{CD}, d_{ED}\} = 8$

- Merge the closest clusters (A and (CE)).

Single linkage: Illustration

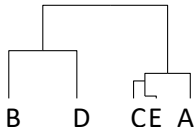
	(ACE)	B	D
(ACE)	0		
B	7	0	
D	6	5	0

	(ACE)	(BD)
(ACE)	0	
(BD)	6	0

- Recompute cluster distances (single linkage)
 $d_{(ACE)B} = \min\{d_{AB}, d_{(CE)B}\} = 7$
 $d_{(ACE)D} = \min\{d_{AD}, d_{(CE)D}\} = 6$
- Merge the closest clusters (B and D).
- Recompute cluster distances (single linkage):
 $d_{(ACE)(BD)} = \min\{d_{(ACE)B}, d_{(ACE)D}\} = 6$
- Merge clusters (ACE) and (BD).

Single linkage: Illustration

- Dendrogram



- Some notes:

- The single linkage algorithm allows for non-symmetrical clusters, but can diverge to meaningless chains
- Complete linkage ensures all elements in a cluster are within some maximum distance of each other, but sensitive to outliers
- Average linkage is a compromise

Divisive hierarchical clustering

- Top to bottom
- Basic concept: apply a “flat” clustering algorithms (e.g. K -means with $K = 2$) recursively
- *Much less attention in literature*

Practical Considerations

Advantages

- No apriori number of clusters required
- Simple algorithms
- Self-organized structural view of data

Disadvantages

- Dendrogram often difficult to visualize
- Difficult to extract “flat” clusters